# CHAPTER 14

# Attitude estimation

## 14.1. Introduction

This chapter describes how to design static attitude estimators using measurements from star cameras, planet sensors, Sun sensors, magnetometers, GPS receivers, and other devices. The algorithms in this chapter produce an estimate based on one set of data, though noise filtering may be employed. Planet sensors, Sun sensors, and magnetometers produce two-axis measurements. A GPS receiver, with suitable antennas, can produce three-axis attitude information. The chapter on Sensors provides additional information.

Chapter 15 shows how to put attitude estimation in a recursive framework. This allows you to rigorously incorporate uncertainty into the attitude estimation process.

## 14.2. Star sensor

First, get unit vectors for each valid star $k$ in the field of view,

$$u_{M_k} = \text{Unit}\left(\begin{bmatrix} \frac{p_{x_k}}{f} \\ \frac{p_{y_k}}{f} \\ 1 \end{bmatrix}\right) \tag{14.1}$$

where $p_x$ and $p_y$ are the focal-plane coordinates and $f$ is the focal length. Fig. 14.1 shows the pinhole-camera optical diagram and the pixel plane. The star images are defocused to spread them among multiple pixels. This allows for subcentroid location accuracy.

At least three centroids are needed. More centroids will reduce the errors due to noise. Find the corresponding catalog unit vectors, $u_C$ through star identification. Make a 3-by-$n$ matrix of the vectors. For example, with four stars the matrix is

$$u_M = \begin{bmatrix} u_{M_1} & u_{M_2} & u_{M_3} & u_{M_4} \end{bmatrix} \tag{14.2}$$

Then, compute the transformation matrix with the pseudoinverse,

$$m = u_M \text{pinv}\,(u_C) \tag{14.3}$$

You then convert $m$ to an attitude quaternion that will orthonormalize it.


https://doi.org/10.1016/B978-0-32-399915-1.00025-5

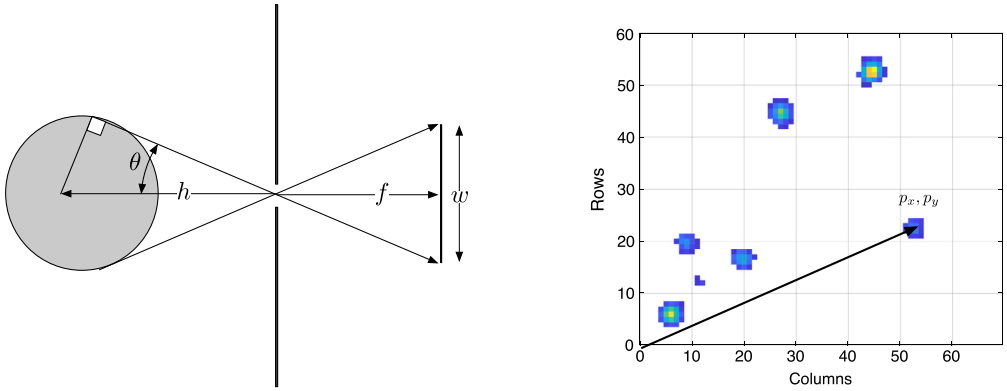



**Figure 14.1** Star-camera geometry. The diagram on the left shows the pinhole camera. The image on the right shows a focal plane.

## 14.3. Planet sensor

### 14.3.1 Acquisition

Chapter 11 describes the mathematical basis for planet-sensor measurements. Both static and scanning sensors must be pointed at the Earth (or planet) to get a valid measurement. Since most sensors have a limited field of view, it is necessary to have an alternative way to find the planet when the sensor cannot see its target. For a momentum-bias satellite, with its momentum vector normal to the orbit, this is just a matter of pitching the spacecraft (rotating around orbit normal) until the planet is in view. For a three-axis satellite, a search may be needed. A magnetometer can provide a two-axis attitude that can be used to find the planet if there is an onboard magnetic-field model and the orbit position is known. A Sun sensor can also be used for this purpose.

### 14.3.2 Roll and pitch measurements from a planet sensor

Static planet sensors have a fixed ring of thermal sensors that skirt the horizon of the planet. The output is proportional to the amount of the sensor that sees the planet. If the sensor sees cold space the output is, ideally, zero. If it is entirely on the planet its output is one. Fig. 14.2 shows a sensor with four detectors 90 degrees apart. The field of view is set by the operational altitude of the sensor. The figure shows two cases; one with zero roll and the other with 0.6 degrees of roll.

The chapter on Machine Intelligence shows how a neural network can be trained to convert sensor outputs into roll and pitch. In this chapter, we create a simple algorithm that uses two sensors for roll and two for pitch.

Example 14.1 computes the output from a static Earth sensor. The last plot is the fit to the data. As can be seen, the outputs of the sensors are linearly related to roll and

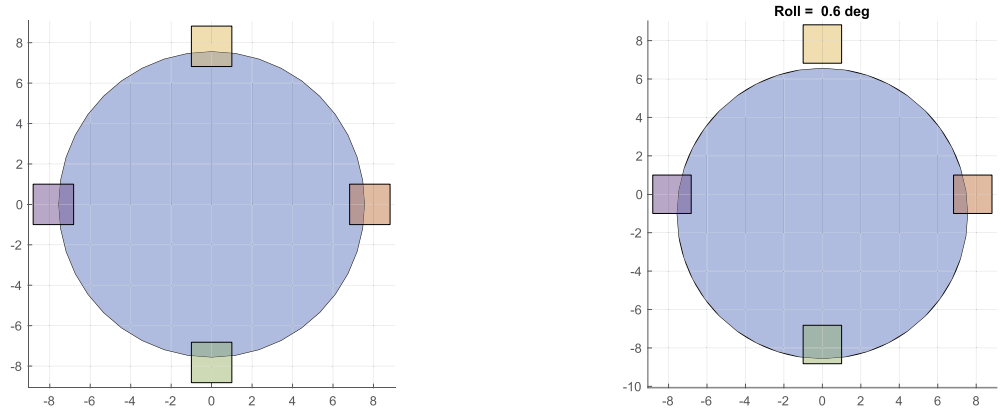


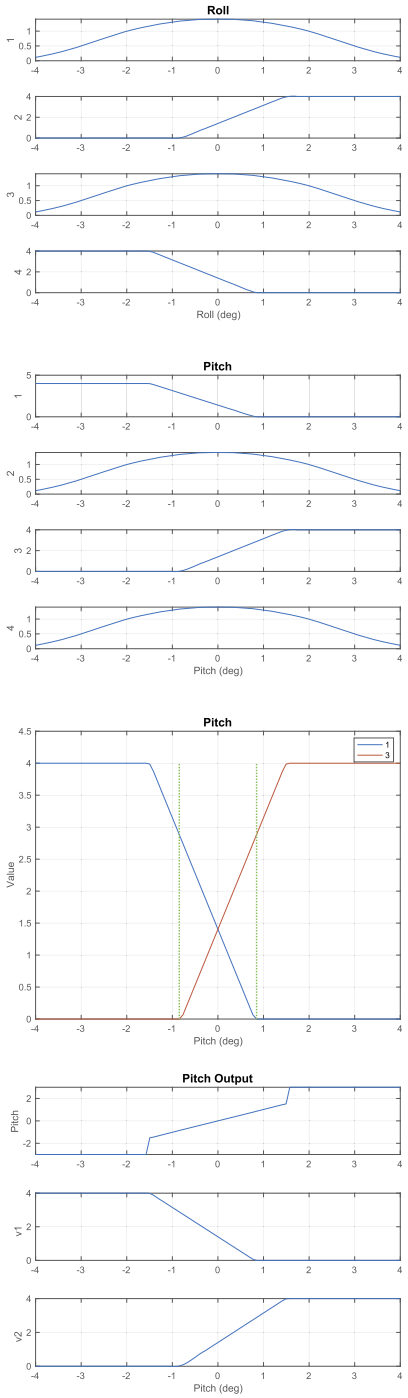

**Figure 14.2** Earth sensor at zero roll and 0.6 deg roll.

pitch so a simple algorithm will suffice. The first plot shows the four sensor outputs for a roll sweep. Sensors 2 and 4 are linearly related to roll when they are on the edge of the Earth. Sensors 1 and 3 give outputs over a range of roll but do not give sign inputs. In addition, the outputs are nonlinear. The second plot shows a similar pattern for pitch, only with sensors 1 and 3. The third plot shows sensors 1 and 3 for pitch. The last plot shows a fit to pitch using sensor 1 and 3 data only.

The fit sets the output to 3 with the sign based on whether the pitch is negative or positive. When both v1 and v2 are valid it averages their outputs.

```
% Out of range
pitch(v1>=3.99)=-3;
pitch(v3>=3.99)= 3;

% 4 is -1.52 2.74 is = +0.79
j = find(v1<3.99);
pitch1(j) = -1.52+(4-v1(j))*(1.52+0.79)/4;

% 4 is -1.52 2.74 is = -0.79
j = find(v3<3.99);
pitch3(j) = 1.52+(v3(j)-4)*(1.52+0.79)/4;

j = find(v1<3.99 & v3 ==0);
pitch(j) = pitch1(j);
j = v3<3.99 & v1 ==0;
pitch(j) = pitch3(j);
j = find(v3 > 0 & v1 > 0);
```

```
%% Earth Sensor

d = StaticEarthSensor;

d.az = [0 pi/2 pi 3*pi/2];
d.el = (pi/20)*ones(1,4);


r    = [0;0;42167];
StaticEarthSensor([1;0;0;0],r,d);


n = 100;
y= zeros(4,n);

q = [1;0;0;0];

% Roll
angle = linspace(-4,4);
for k = 1:n
  q       = Sa2Q([angle(k)*pi/180;0;0]);
  y(:,k) = StaticEarthSensor(q,r,d);
end

Plot2D(angle,y,'Roll (deg)', {'1' '2' '3' '4'}, '
       Roll')

roll = 0.01;
StaticEarthSensor(QUnit([1;roll;0;0]),r,d);
s = sprintf('Roll = %4.1f deg',roll*180/pi);
title(s)

% Pitch
for k = 1:n
  q       = Sa2Q([0;angle(k)*pi/180;0;0]);
  y(:,k) = StaticEarthSensor(q,r,d);
end

Plot2D(angle,y,'Pitch (deg)', {'1' '2' '3' '4'},
       'Pitch')
Plot2D(angle,y([1 3],:),'Pitch (deg)', {'Value'},
       'Pitch')

x = [-0.848 -0.848];
line(x,[0 4],'linestyle',':','linewidth',2,'color
       ',[0 1 0])

x = [0.848 0.848];
line(x,[0 4],'linestyle',':','linewidth',2,'color
       ',[0 1 0])
legend('1','3')

p = SensorToPitch(y(1,:),y(3,:));
Plot2D(angle,[p;y([1 3],:)],'Pitch (deg)',{'Pitch
       ' 'v1' 'v2'},'Pitch Output')
```




**Example 14.1:** Static Earth-sensor outputs and a pitch-angle fit to outputs 1 and 3.

```
pitch(j) = 0.5*(pitch1(j) + pitch3(j));
```

A similar algorithm, using v2 and v3 can be used for roll.

## 14.4. Sun sensor

Since a single sensor is just a cosine detector it cannot give direction. If two such sensors are mounted together as shown in Fig. 14.3 the direction can be found. Assume that

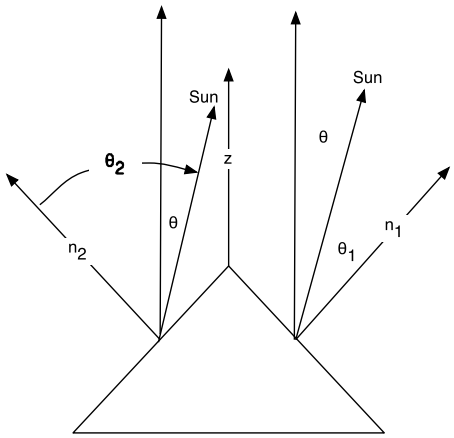



**Figure 14.3** Simple one-dimensional Sun sensor.

the output of the sensor is

$$y = \cos\theta \tag{14.4}$$

The angles are

$$\theta_1 = \cos^{-1}(n_1^T u_{Sun}) \tag{14.5}$$
$$\theta_2 = \cos^{-1}(n_2^T u_{Sun}) \tag{14.6}$$

If the angle between $z$ and the sensor normal is $\beta$, then

$$\theta_1 = \theta + \beta \tag{14.7}$$
$$\theta_2 = \beta - \theta \tag{14.8}$$

so that

$$\theta = \frac{\theta_2 - \theta_1}{2} \tag{14.9}$$

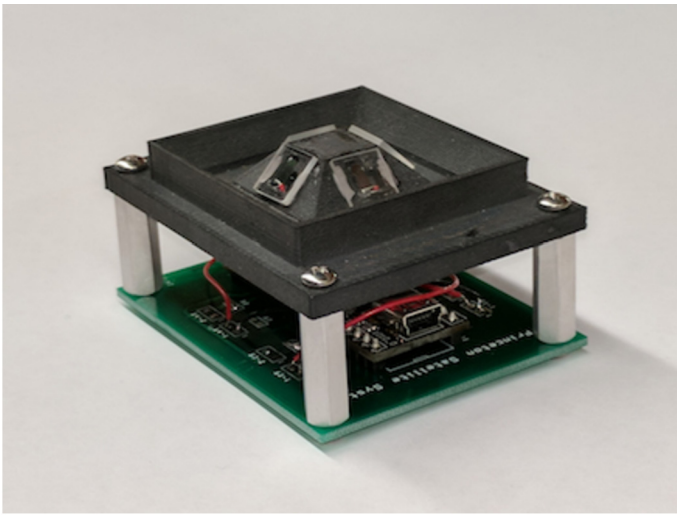


**Figure 14.4** Two-axis analog Sun sensor. The supporting structure is 3D printed. The windows are visible on the top.

which gives sign information. Fig. 14.4 shows a pyramidal sensor that can measure two axes. This sensor has an analog-to-digital converter and uses USB as its digital interface.

More generally, the output of a sensor is modeled by a cosine series

$$y = \sum_{k=1}^{N} a_k \cos^k \theta \tag{14.10}$$

where $a_k$ is specified by the vendor. If $a_k$ for $k > 1$ are small, this can be solved using a Newton–Raphson algorithm to find the root of $f(x) = 0$. The algorithm is iterative.

$$\delta = -\frac{f(x)}{f'(x)} \tag{14.11}$$

$$x = x + \delta \tag{14.12}$$

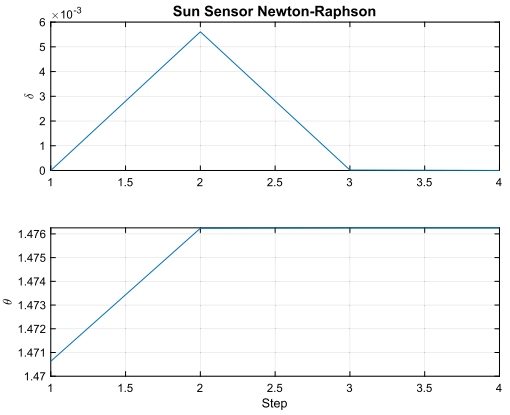
Continue this until $\delta$ is small. In this case

$$f(\theta) = y - \sum_{k=1}^{N} a_k \cos^k \theta \tag{14.13}$$

$$f'(\theta) = -\sum_{k=1}^{N} k a_k \sin\theta \cos^{k-1} \theta \tag{14.14}$$

Example 14.2 shows how quickly the Newton–Raphson method converges. As is shown, one iteration is sufficient.

```
a = [1 0.6 0.3];
y = 0.1;
n = 4;

delta    = zeros(1,n);
theta    = zeros(1,n);
theta(1) = acos(y)/a(1);

for k = 2:n
  c        = cos(theta(k-1));
  s        = sin(theta(k-1));
  f        = y - (a(1)*c + a(2)*c^2 + a(3)*c^3);
  fD       = s*(a(1) + 2*a(2)*c + 3*a(3)*c^2);
  delta(k) = -f/fD;
  theta(k) = theta(k-1) + delta(k);
end

Plot2D(1:n,[delta;theta],'Step',{'\delta' '\theta'},'Sun Sensor Newton-Raphson')
set(gca,'xlim',[1 n]);

c = cos(theta(end));
s = sin(theta(end));
err = y - (a(1)*c + a(2)*c^2 + a(3)*c^3)
```




**Example 14.2:** Newton–Raphson method for Sun-angle computation.

## 14.5. Magnetometer

A magnetometer gives the magnetic-field vector in the body frame. If an ephemeris is available with a magnetic-field model, then the angle between the measured vector and the ECI magnetic-field vector can be found. In Section 14.7 we show how to combine the magnetic-field vector with another vector to get the ECI to body quaternion. For just the angle you first compute the magnetic field for the current orbital position, transform that into the ECI frame and then take the inverse cosine of the dot product. The process for getting the ephemeris magnetic field vector is shown in Fig. 14.5.

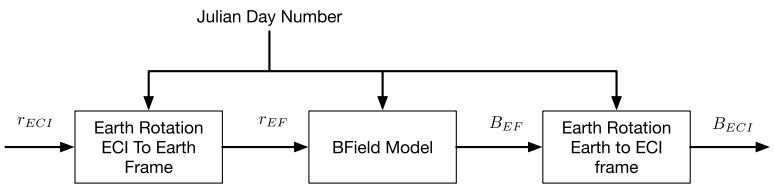



**Figure 14.5** ECI magnetic-field computation.

The angle is

$$\theta = \cos^{-1}(u_{ECI}^T u_{body}) \tag{14.15}$$

This defines a cone around the magnetic field vector. Another measurement is needed to find the location on the cone.

## 14.6. GPS

GPS and other navigation satellites send signals that are used to determine position and velocity. Each signal gives range and range rate. Four signals are needed to resolve the three-dimensional position and velocity. This same system can be used for attitude determination [1,2]. The concept is shown in Fig. 14.6.

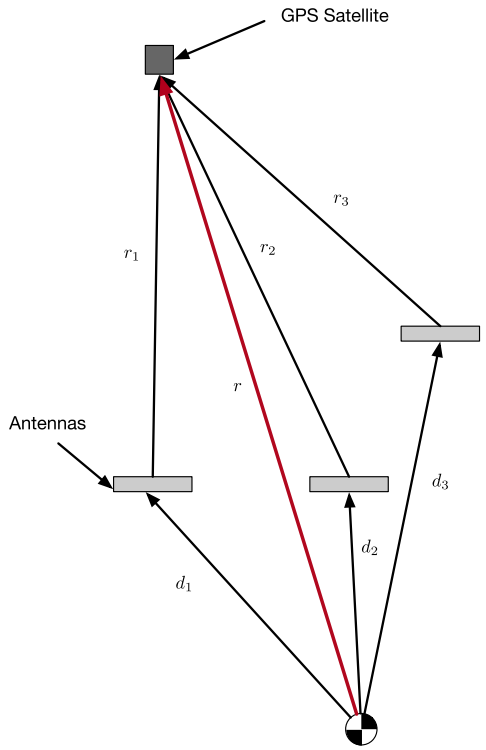



**Figure 14.6** GPS attitude geometry. The range signals will be a function of the attitude. Multiple GPS satellites are used.

GPS attitude determination uses the time difference of the double-differenced carrier-phase measurements of the GPS signals. The attitude precision that can be achieved from a GPS attitude determination system is limited by the multipath signals and the baseline length between the antennas [3]. For this analysis, we will assume that the measurements of $r$ and $r_k$ have sufficient precision. The equation for each vector for the $j$th GPS satellite is

$$r_j + A(r_{jk} + d_k) = 0 \tag{14.16}$$

where $A$ transforms from the inertial to the body frame. $A$ is the attitude matrix we wish to compute. $r$ is known from the GPS orbit determination. $d_k$ is also known. The

measurements are just out of range, so

$$r_{jk} = \rho_{jk} A u_{jk} \tag{14.17}$$

where $\rho_k$ is the measured range and $u_k$ is the unit vector pointing from the antenna to the GPS satellite measured in the body frame. The vector equation is now

$$A(r_{jk} + d_k) = r_j \tag{14.18}$$

Assume that all of the vectors are aligned with $r$ and the unit vector for $r$ is $u$. Then, the equation becomes

$$A(\rho_{jk} u_j + d_k) = r_j \tag{14.19}$$

If we have at least three measurements, we can compute $A$. Let $b_{kj} = \rho_{kj} u_j + d_k$.

$$A = F\text{pinv}(B) \tag{14.20}$$

where the columns of $B$ are $b_{kj}$ and for each column $kj$ of $B$ there is a corresponding column of $F$ that is $r_j$. $A$ is most easily orthonormalized by converting it to a quaternion. Example 14.3 gives an example. You need to have multiple satellites for a good solution, as shown in the script. ans is the true quaternion. This is not an issue since GPS requires at least four satellites for accurate orbit determination.

```
q    = QRand;
a    = Q2Mat(q);

rGPS = 22000*Unit(randn(3,5));

rSat = [0;0;7000];
d    = [1 0 -1;1 -1 1;0 0 0];

r    = rGPS - rSat;
i    = 0;

b    = zeros(3,3);
for j = 1:size(r,2)
  rS = a'*r(:,j);
  u  = Unit(rS);
  for k = 1:3
   i      = i + 1;
   rK     = rS - d(:,k);
   rho    = Mag(rK);
   b(:,i) = rho*u + d(:,k);
   f(:,i) = r(:,j);
 end
end

aC = f*pinv(b);

% Orthnormalze
Mat2Q(aC)
q
```

```
ans =
1.921206024595309e-01   -
2.403235884576184e-01
5.256707101347798e-01   -
7.930965479134801e-01
q =
1.921221216347452e-01   -
2.403228320128406e-01
5.256708682857887e-01   -
7.930978281571304e-01
```

**Example 14.3:** GPS attitude solution.

## 14.7. Earth/Sun/magnetic field

It is possible to measure the ECI to body quaternion using two vector measurements from a combination of Earth sensors, Sun sensors, and magnetometers. Fig. 14.7 shows the geometry.

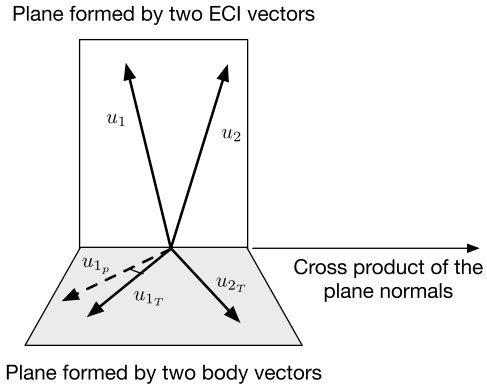



**Figure 14.7** Vector geometry for Earth/Sun/magnetic-field-attitude estimation.

First, unitize the measured and cataloged vectors. Then, compute the normals for each plane.

$$n = u_1 \times u_2 \tag{14.21}$$

$$n_T = u_{1_T} \times u_{2_T} \tag{14.22}$$

Unitize both $n$ and $n_T$. Create a quaternion that rotates $n$ into $n_B$,

$$a = n \times n_T \tag{14.23}$$

$$d = n^T n_T \tag{14.24}$$

$$s = \sqrt{2(1+d)} \tag{14.25}$$

$$q_T = \begin{bmatrix} \frac{1}{2}s \\ \frac{c}{s} \end{bmatrix} \tag{14.26}$$

$$u_{1_P} = q \otimes u_1 \tag{14.27}$$

Create a quaternion that rotates $u_{1_P}$ into $u_{1_T}$,

$$a = u_{1_P} \times u_{1_T} \tag{14.28}$$

$$d = u_{1_P}^T u_{1_T} \tag{14.29}$$

$$s = \sqrt{2(1+d)} \tag{14.30}$$

$$q_P = \begin{bmatrix} \frac{1}{2}s \\ \frac{c}{s} \end{bmatrix} \tag{14.31}$$

The ECI to body quaternion is

$$q = q_T \otimes q_P \tag{14.32}$$

Any combination of nadir vectors, magnetic-field vectors, and Sun vectors can be used. You need to know the ECI references for these vectors. The body-unit vectors can be obtained from a magnetometer and any two-axis Earth or Sun sensor. The nadir vector can be obtained from GPS or an onboard ephemeris.

## 14.8. Noise filters

Sensors can be noisy and may need filtering. The order of a filter determines how effective it is at removing noise. The higher the order, the faster the roll-off. This comes with the price of increased lag in the loop. If the cutoff frequency is near the control bandwidth, it will be necessary to adjust the control-algorithm parameters.

Let us work in the continuous domain. We use a proportional-derivative controller with a first-order filter. The PD controller is

$$\frac{u}{\theta} = K(1 + \tau_r s) \tag{14.33}$$

where $K$ is the forward gain and $\tau_r$ is the derivative time constant. The first-order filter is

$$\frac{\theta}{\theta_m} = \frac{1}{\tau_f s + 1} \tag{14.34}$$

At low frequencies this has a gain of 1. At high frequencies the filter is

$$\frac{\theta}{\theta_m} = \frac{1}{\tau_f s} \tag{14.35}$$

The response decreases linearly as $s$ increases. This can also be written in state-space form

$$\dot{x} = -\frac{1}{\tau_f}x + \frac{1}{\tau_f}u \tag{14.36}$$

$$y. = x \tag{14.37}$$

The total transfer function is the series of the first-order filter and the PD controller.

$$\frac{u}{\theta_m} = K\left(\frac{1 + \tau_r s}{1 + \tau_f s}\right) \tag{14.38}$$

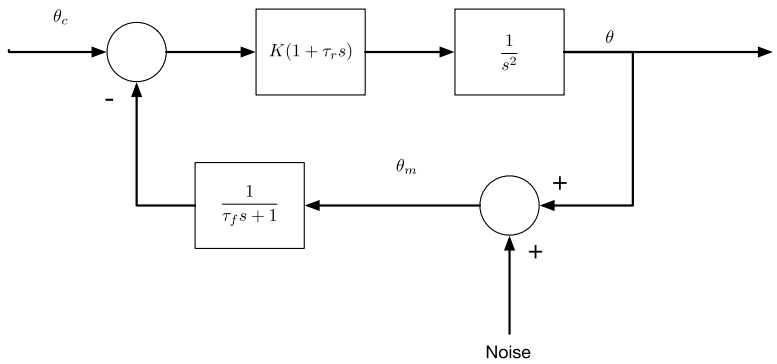
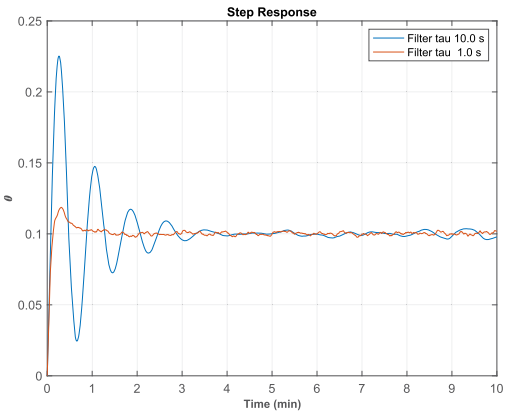



**Figure 14.8** The closed-loop system block diagram.

```
%% First order filter

omegaN    = 0.1;
zeta      = 2;
tauR      = 2*zeta/omegaN;
kF        = omegaN^2;
thetaC    = 0.1;
oneSigma  = 0.01;
tauF      = tauR*[0.5  0.05];
n         = 6000;
dT        = 0.1;
xP        = zeros(2,n);
s         = cell(2);

for j = 1:2
  d.tauF = tauF(j);
  s{j}   = sprintf('Filter tau %4.1f s',d.tauF);
  x      = [0;0];
  errOld = 0;
  thetaF = 0;
  for k = 1:n
    thetaM    = x(1) + oneSigma*randn;
    thetaF    = thetaF + dT*(thetaM - thetaF)/
          tauF(j);
    err       = thetaF - thetaC;
    u         = -kF*(err + tauR*(err-errOld)/dT);
    errOld    = err;
    xP(j,k)   = x(1);
    x         = RK4(@RHS,x,dT,0,u);
  end
end

t = (0:n-1)*dT;

TimeHistory(t,xP,{'\theta'},'Step Response'
      ,{1:2});
legend(s{1},s{2})

%% Right hand side
function xDot = RHS(x,~,u)
omega     = x(2);
xDot      = [omega;u];
end
```




**Example 14.4:** First-order filter response for two different filter time constants.

Assume we want a damping ratio of 1 and a bandwidth of 0.1 rad/s. Assume that the plant is a double integrator, $\frac{1}{s^2}$, then $K = \omega_n^2$ and $\tau_r = \frac{2}{\omega_n}$. Let us look at the controller response with a filter cutoff at twice the control-system bandwidth and five times the control-system bandwidth. The system block diagram is shown Fig. 14.8.

Example 14.4 shows the response with two different first-order filter time constants. The controller and filter are implemented with a first-order hold. The filter with the smaller time constant, and higher filter cutoff, passes more noise but has better damping. The filter with the larger time constant has very low damping but the signal is cleaner.

An $n$th-order Butterworth filter is often a reasonable choice for noise filtering. Butterworth filters are called "maximally flat" filters because they do not have any ripple in the stop band or the pass band. This makes them well suited for control systems. Peaking is not always undesirable, because it reduces the phase lag. An even order (2nd, 4th etc.) can be assembled by putting 2nd-order state-space blocks in series. One block is

$$\dot{x} = ax + bu \tag{14.39}$$

$$y = cx + du \tag{14.40}$$

$$a = \begin{bmatrix} 0 & \omega_C \\ -\omega_C & -2\zeta\omega_C \end{bmatrix} \tag{14.41}$$

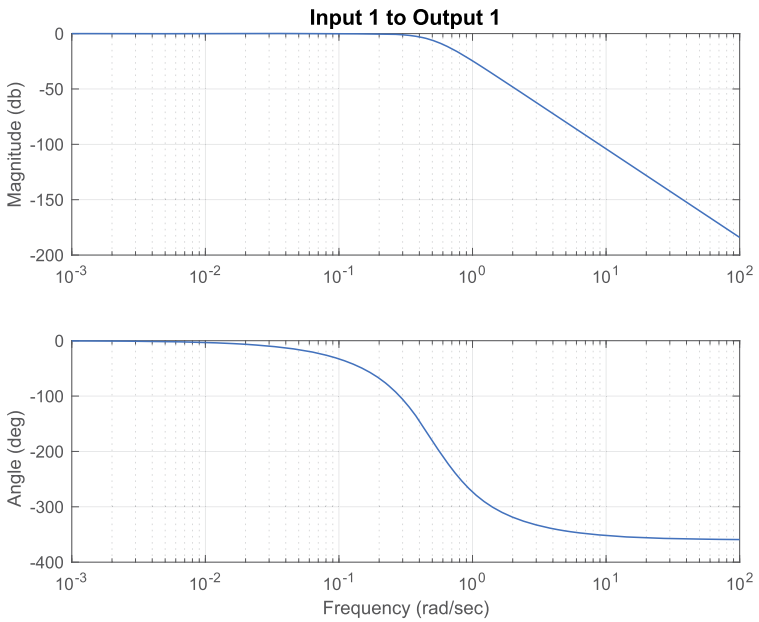



**Figure 14.9** Butterworth fourth-order filter with $\omega_C = 0.5$.

$$b = \begin{bmatrix} 0 \\ \omega_C \end{bmatrix} \tag{14.42}$$

$$c = \begin{bmatrix} 1 & 0 \end{bmatrix} \tag{14.43}$$

$$d = 0 \tag{14.44}$$

where $\zeta = \frac{\sqrt{2}}{2}$ and $\omega_C$ is the cutoff frequency. An example of a fourth-order Butterworth filter, found by calling CButter( 4, 0.5, 1 ) is shown in Fig. 14.9.

## References

[1] Charles Wang, Rodney A. Walker, Werner Enderle, Single antenna attitude determination for FedSat, in: Proceedings Institute of Navigation GPS-02, 2002, pp. 1–12.
[2] G. Lu, Development of a GPS Multi-Antenna System for Attitude Determination, Ph.D. thesis, University of Calgary, 1995.
[3] D. Kim, R. Langley, GPS RTK-Based Attitude Determination for the e-POP Platform onboard the Canadian CASSIOPE Spacecraft in Low Earth Orbit, 01 2007.